Lecture 9: Dynamic programming and numerical methods in function spaces

ECON 7510

Cornell University

Adam Harris

Dynamic programming

Dynamic model: ?

Dynamic programming

Dynamic model: Model in which the current action affects the future state.

Dynamic programming

Dynamic model: Model in which the current action affects the future state.

Dynamic programming: **?**

Dynamic programming

Dynamic model: Model in which the current action affects the future state.

Dynamic programming: A way of solving dynamic problems that involves solving the problem in subparts, using recursion.

Dynamic programming

Dynamic model: Model in which the current action affects the future state.

Dynamic programming: A way of solving dynamic problems that involves solving the problem in subparts, using recursion.

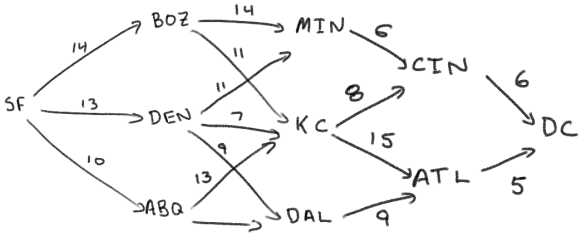
Today's plan: Solving dynamic programming problems that are:

1. Discrete-state, deterministic payoffs
2. Discrete-state, stochastic payoffs
3. Continuous-state, stochastic payoffs

Context: Today, we solve a model with known parameters. In a few lectures, we'll be *estimating* model parameters.

Motivating example: Route choice

Figure: Choosing a route from San Francisco to DC



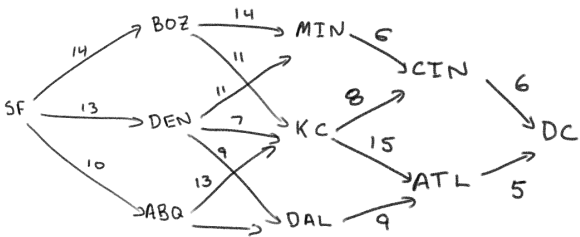
Each edge has a different cost (travel time)

Question: How should we go about determining the best route?

- Suppose brute force (evaluating every possible route) is not an option

Motivating example: Route choice

Figure: Choosing a route from San Francisco to DC



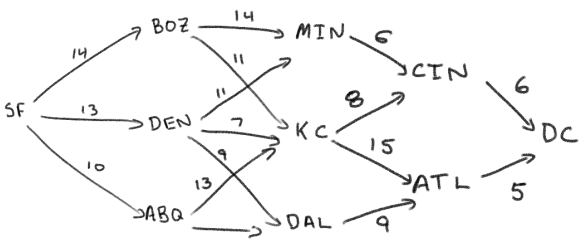
Each edge has a different cost (travel time)

Question: How should we go about determining the best route?

- Suppose brute force (evaluating every possible route) is not an option
- Backward induction

Motivating example: Route choice

Figure: Choosing a route from San Francisco to DC



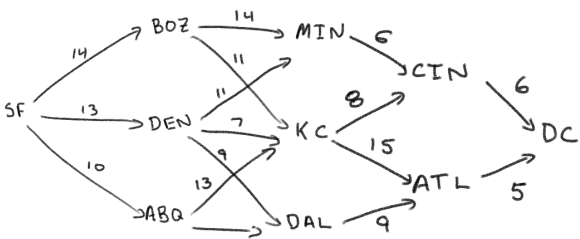
Each edge has a different cost (travel time)

Question: How should we go about determining the best route?

- Suppose brute force (evaluating every possible route) is not an option
- Backward induction
- When making a decision at step t, assume that all future decisions at $t+1, t+2, \ldots$ will be optimal

Motivating example: Route choice

Figure: Choosing a route from San Francisco to DC



Each edge has a different cost (travel time)

Question: How should we go about determining the best route?

- Suppose brute force (evaluating every possible route) is not an option
- Backward induction
- When making a decision at step t, assume that all future decisions at $t+1, t+2, \ldots$ will be optimal

Question: What if there are infinitely many time periods/steps? How does that change things?

Dynamic programming: Finite versus infinite horizon

- **Finite-horizon**:
  - → Solve by backwards induction (BI).
- **Infinite-horizon**:
  - → Can *approximate* solution using BI. **Question**: How?

Dynamic programming: Finite versus infinite horizon

- **Finite-horizon**:
  - → Solve by backwards induction (BI).
- **Infinite-horizon**:
  - → Can *approximate* solution using BI. **Question**: How?
  - → Solve exactly by value function iteration (VFI).

Dynamic programming: Finite versus infinite horizon

- **Finite-horizon**:
  - → Solve by backwards induction (BI).
- **Infinite-horizon**:
  - → Can *approximate* solution using BI. **Question**: How?
  - → Solve exactly by value function iteration (VFI).
- **Note**: Can also solve finite-horizon problems using VFI, though BI is usually more computationally efficient

Dynamic programming: Deterministic payoffs, discrete-state example

- Time is discrete, $t = 0, 1, 2, \ldots$
- Discount factor: δ
- Discrete state: $x \in \{1, 2, \ldots, N\}$
- Binary choice: $a \in \{0, 1\}$
- Static payoff: $u_a(x)$
- Transition matrices: $M^a \in \mathbb{R}^{N \times N}$ where $M^a_{xx'} = Pr(x_{t+1} = x' \mid a_t = a, x_t = x)$

Dynamic programming: Deterministic payoffs, discrete-state example

So payoff from action a, inclusive of dynamic term, is

$$v_a(x_t) = u_a(x_t) + \delta \mathbb{E}[V(x_{t+1}) \mid x_t, a_t]$$

$$= u_a(x_t) + \delta \sum_{x'} M^a_{x_t, x'} V(x')$$

where V is the value function:

Dynamic programming: Deterministic payoffs, discrete-state example

So payoff from action a, inclusive of dynamic term, is

$$\begin{aligned} v_a(x_t) &= u_a(x_t) + \delta \mathbb{E}[V(x_{t+1}) \mid x_t, a_t] \\ &= u_a(x_t) + \delta \sum_{x'} M^a_{x_t, x'} V(x') \end{aligned}$$

where V is the value function:

$$\begin{aligned} V(x) &= \max\{v_0(x), v_1(x)\} \\ &= \max\{u_0(x) + \delta \mathbb{E}[V(x_{t+1}) \mid x_t = x, a_t = 0], u_1(x) + \delta \mathbb{E}[V(x_{t+1}) \mid x_t = x, a_t = 1]\} \end{aligned}$$

The above is a fixed-point problem in $V(x)$.

Dynamic programming: Deterministic payoffs, discrete-state example

$$
\begin{aligned}
V(x) &= \max\{v_0(x), v_1(x)\} \\
&= \max\{u_0(x) + \delta \mathbb{E}[V(x_{t+1}) \mid x_t = x, a_t = 0], u_1(x) + \delta \mathbb{E}[V(x_{t+1}) \mid x_t = x, a_t = 1]\}
\end{aligned}
$$

The above is a fixed-point problem in $V(x)$.

Since x is discrete can think of u_0, u_1, V as a vectors rather than functions:

$$
V = \begin{bmatrix} V(1) \\ V(2) \\ \vdots \\ V(N) \end{bmatrix}
$$

So our goal is to solve a nonlinear fixed point equation for the vector V.

Dynamic programming: Deterministic, discrete-state example

For $k = 1, 2, \ldots$ until $\left| V^k - V^{k-1} \right| < \text{TOL}$

$$V_i^{k+1} = \max_{a \in \{0,1\}} \left[u_{ai} + \delta M_i^a \cdot V^k \right] \quad \text{for each } i = 1, \ldots N$$

Dynamic programming: Deterministic, discrete-state example

For $k = 1, 2, \ldots$ until $\left| V^k - V^{k-1} \right| < \text{TOL}$

$$V_i^{k+1} = \max_{a \in \{0,1\}} \left[u_{ai} + \delta M_i^a \cdot V^k \right] \quad \text{for each } i = 1, \ldots N$$

See `L9.py`.

Dynamic programming: Stochastic payoffs, discrete-state example

- Time is discrete, $t = 0, 1, 2, \ldots$
- Discount factor: δ
- Discrete state: $x \in \{1, 2, \ldots, N\}$
- Binary choice: $a \in \{0, 1\}$
- Static payoff: $u_a(x) + \epsilon_{at}$ where $\epsilon_{at} \sim$ iid F
- Transition matrices: $M^a \in \mathbb{R}^{N \times N}$ where $M^a_{xx'} = Pr(x_{t+1} = x' \mid a_t = a, x_t = x)$

Dynamic programming: Stochastic payoffs, discrete-state example

Recall: Before we had

$$V(x) = \max\{v_0(x), v_1(x)\}$$

where

$$\begin{aligned} v_a(x) &= u_a(x) + \delta \mathbb{E}[V(x_{t+1}) \mid x_t = x, a_t] \\ &= u_a(x) + \delta \sum_{x'} M^a_{x,x'} V(x') \end{aligned}$$

and

$$a^*(x) = 1 \Leftrightarrow v_1(x) > v_0(x)$$

Question: What changes with the introduction of ϵ? [Hint about notation: Let $v_a(x_t)$ be payoff excluding ϵ_{at}.]

Dynamic programming: Stochastic payoffs, discrete-state example

Recall: Before we had

$$V(x) = \max\{v_0(x), v_1(x)\}$$

where

$$\begin{aligned} v_a(x) &= u_a(x) + \delta \mathbb{E}[V(x_{t+1}) \mid x_t = x, a_t] \\ &= u_a(x) + \delta \sum_{x'} M^a_{x,x'} V(x') \end{aligned}$$

and

$$a^*(x, \epsilon) = 1 \Leftrightarrow v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0$$

Question: What changes with the introduction of ϵ? [Hint about notation: Let $v_a(x_t)$ be payoff excluding ϵ_{at}.]

Dynamic programming: Stochastic payoffs, discrete-state example

Recall: Before we had

$$V(x) = \max\{v_0(x), v_1(x)\}$$

where

$$\begin{aligned} v_a(x) &= u_a(x) + \delta \mathbb{E}[V(x_{t+1}) \mid x_t = x, a_t] \\ &= u_a(x) + \delta \sum_{x'} M^a_{x,x'} V(x') \end{aligned}$$

and

$$a^*(x, \epsilon) = 1 \Leftrightarrow v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0$$

Question: What changes with the introduction of ϵ? [Hint about notation: Let $v_a(x_t)$ be payoff excluding ϵ_{at}.]

Ex-ante expected value function (before ϵ is realized):

$$EV(x) = \mathbb{E}\left[\max\{v_0(x) + \epsilon_0, v_1(x) + \epsilon_1\}\right]$$

Dynamic programming: Stochastic payoffs, discrete-state example

Recall: Before we had

$$V(x) = \max\{v_0(x), v_1(x)\}$$

where

$$\begin{aligned} v_a(x) &= u_a(x) + \delta \mathbb{E}[EV(x_{t+1}) \mid x_t = x, a_t] \\ &= u_a(x) + \delta \sum_{x'} M^a_{x,x'} EV(x') \end{aligned}$$

and

$$a^*(x, \epsilon) = 1 \Leftrightarrow v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0$$

Question: What changes with the introduction of ϵ? [Hint about notation: Let $v_a(x_t)$ be payoff excluding ϵ_{at}.]

Ex-ante expected value function (before ϵ is realized):

$$EV(x) = \mathbb{E}\left[\max\{v_0(x) + \epsilon_0, v_1(x) + \epsilon_1\}\right]$$

Dynamic programming: Stochastic payoffs, discrete-state example

Ex-ante expected value function

$$EV(x) = \mathbb{E}\left[\max\{v_0(x) + \epsilon_0, v_1(x) + \epsilon_1\}\right]$$

Recall: We know the distribution F of ϵ.

Question: If we knew $v_0(x)$, $v_1(x)$, how could we compute $EV(x)$?

Dynamic programming: Stochastic payoffs, discrete-state example

Ex-ante expected value function

$$EV(x) = \mathbb{E}\left[\max\{v_0(x) + \epsilon_0, v_1(x) + \epsilon_1\}\right]$$

Recall: We know the distribution F of ϵ.

Question: If we knew $v_0(x)$, $v_1(x)$, how could we compute $EV(x)$?

- Expectation = integration, so can use quadrature or random sampling.

Dynamic programming: Stochastic payoffs, discrete-state example

Ex-ante expected value function

$$EV(x) = \mathbb{E}\left[\max\{v_0(x) + \epsilon_0, v_1(x) + \epsilon_1\}\right]$$

Recall: We know the distribution F of ϵ.

Question: If we knew $v_0(x)$, $v_1(x)$, how could we compute $EV(x)$?

- Expectation = integration, so can use quadrature or random sampling.
- Maybe for some F, there is a closed form expression for $EV(x)$ in terms of $v_0(x)$, $v_1(x)$?

Dynamic programming: Stochastic payoffs, discrete-state example

Policy function

$$a^*(x, \epsilon) = 1 \Leftrightarrow v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0$$

Recall: We know the distribution F of ϵ.

Question: If we know $v_0(x)$, $v_1(x)$, how can we compute $p(x) = \Pr(a^*(x, \epsilon) = 1 \mid x)$?

Dynamic programming: Stochastic payoffs, discrete-state example

Policy function

$$a^*(x, \epsilon) = 1 \Leftrightarrow v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0$$

Recall: We know the distribution F of ϵ.

Question: If we know $v_0(x)$, $v_1(x)$, how can we compute $p(x) = \Pr\left(a^*(x, \epsilon) = 1 \mid x\right)$?

$$\begin{aligned} p(x) &= \Pr\left(a^*(x, \epsilon) = 1 \mid x\right) \\ &= \Pr\left(v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0\right) \\ &= \Pr\left(\epsilon_0 - \epsilon_1 < v_1(x) - v_0(x)\right) \end{aligned}$$

Dynamic programming: Stochastic payoffs, discrete-state example

Policy function

$$a^*(x, \epsilon) = 1 \Leftrightarrow v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0$$

Recall: We know the distribution F of ϵ.

Question: If we know $v_0(x)$, $v_1(x)$, how can we compute $p(x) = \Pr\left(a^*(x, \epsilon) = 1 \mid x\right)$?

$$\begin{aligned} p(x) &= \Pr\left(a^*(x, \epsilon) = 1 \mid x\right) \\ &= \Pr\left(v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0\right) \\ &= \Pr\left(\epsilon_0 - \epsilon_1 < v_1(x) - v_0(x)\right) \end{aligned}$$

— Only certain distributions F give rise to a closed for expression for $p(x)$ in terms of $v_0(x)$, $v_1(x)$?

Dynamic programming: Stochastic payoffs, discrete-state example

One distribution commonly assumed for dynamic discrete choice models gives closed-form expression for both.

Question: Which distribution?

Dynamic programming: Stochastic payoffs, discrete-state example

One distribution commonly assumed for dynamic discrete choice models gives closed-form expression for both.

Question: Which distribution? $\epsilon_{at} \sim$ iid Extreme Value (θ)

Choice probabilities:

$$\begin{aligned} a^*(x, \epsilon) = 1 &\Leftrightarrow v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0 \\ &\Leftrightarrow \epsilon_0 - \epsilon_1 < v_1(x) - v_0(x) \end{aligned}$$

Dynamic programming: Stochastic payoffs, discrete-state example

One distribution commonly assumed for dynamic discrete choice models gives closed-form expression for both.

Question: Which distribution? $\epsilon_{at} \sim$ iid Extreme Value (θ)

Choice probabilities:

$$\begin{aligned} a^*(x, \epsilon) = 1 &\Leftrightarrow v_1(x) + \epsilon_1 > v_0(x) + \epsilon_0 \\ &\Leftrightarrow \epsilon_0 - \epsilon_1 < v_1(x) - v_0(x) \end{aligned}$$

$\epsilon_0 - \epsilon_1 \sim$ Logistic(θ), so

$$\begin{aligned} p(x) &= \Pr\left(\epsilon_0 - \epsilon_1 < v_1(x) - v_0(x)\right) \\ &= \Lambda\left(\theta\left[v_1(x) - v_0(x)\right]\right) \end{aligned}$$

Dynamic programming: Stochastic payoffs, discrete-state example

One distribution commonly assumed for dynamic discrete choice models gives closed-form expression for both.

Question: Which distribution? $\epsilon_{at} \sim$ iid Extreme Value (θ)

Ex-ante expected value function:

$$\begin{aligned} EV(x) &= \mathbb{E}\left[\max\{v_0(x) + \epsilon_0, v_1(x) + \epsilon_1\}\right] \\ &= \frac{1}{\theta} \log\left[\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right)\right] \end{aligned}$$

Dynamic programming: Stochastic payoffs, discrete-state example

One distribution commonly assumed for dynamic discrete choice models gives closed-form expression for both.

Question: Which distribution? $\epsilon_{at} \sim$ iid Extreme Value (θ)

Ex-ante expected value function:

$$\begin{aligned} EV(x) &= \mathbb{E}\left[\max\{v_0(x) + \epsilon_0, v_1(x) + \epsilon_1\}\right] \\ &= \frac{1}{\theta} \log\left[\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right)\right] \end{aligned}$$

Aside: In some applications, both $v_0(x)$ and $v_1(x)$ may be large and negative.
Question: What numerical issue arises here?

Dynamic programming: Stochastic payoffs, discrete-state example

One distribution commonly assumed for dynamic discrete choice models gives closed-form expression for both.

Question: Which distribution? $\epsilon_{at} \sim$ iid Extreme Value (θ)

Ex-ante expected value function:

$$\begin{aligned} EV(x) &= \mathbb{E}\left[\max\{v_0(x) + \epsilon_0, v_1(x) + \epsilon_1\}\right] \\ &= \frac{1}{\theta} \log\left[\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right)\right] \end{aligned}$$

Aside: In some applications, both $v_0(x)$ and $v_1(x)$ may be large and negative.
Question: What numerical issue arises here?

$\rightarrow$ $\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right)$ gets rounded to zero $\rightarrow$ can't take log of zero.

Dynamic programming: Stochastic payoffs, discrete-state example

One distribution commonly assumed for dynamic discrete choice models gives closed-form expression for both.

Question: Which distribution? $\epsilon_{at} \sim$ iid Extreme Value (θ)

Ex-ante expected value function:

$$\begin{aligned} EV(x) &= \mathbb{E}\left[\max\{v_0(x) + \epsilon_0, v_1(x) + \epsilon_1\}\right] \\ &= \frac{1}{\theta} \log\left[\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right)\right] \\ &= v_0(x) + \frac{1}{\theta} \log\left[1 + \exp\left(\theta\left(v_1(x) - v_0(x)\right)\right)\right] \\ &= v_1(x) + \frac{1}{\theta} \log\left[1 + \exp\left(\theta\left(v_0(x) - v_1(x)\right)\right)\right] \end{aligned}$$

Aside: In some applications, both $v_0(x)$ and $v_1(x)$ may be large and negative.
Question: What numerical issue arises here?

$\rightarrow$ $\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right)$ gets rounded to zero $\rightarrow$ can't take log of zero.

Dynamic programming: Stochastic payoffs, discrete-state example

For $k = 1, 2, \ldots$ until convergence

$$\begin{aligned} EV_i^{k+1} &= \frac{1}{\theta} \log \left[\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right) \right] \\ &= \frac{1}{\theta} \log \left[\sum_{a \in \{0,1\}} \exp\left(\theta \left[u_{ai} + \delta M_i^a \cdot EV^k \right]\right) \right] \text{ for each } i = 1, \ldots, N \end{aligned}$$

Now a fixed point problem in terms of EV rather than V.

Dynamic programming: Stochastic payoffs, discrete-state example

For $k = 1, 2, \ldots$ until convergence

$$
\begin{aligned}
EV_i^{k+1} &= \frac{1}{\theta} \log \left[\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right) \right] \\
&= \frac{1}{\theta} \log \left[\sum_{a \in \{0,1\}} \exp\left(\theta \left[u_{ai} + \delta M_i^a \cdot EV^k \right] \right) \right] \text{ for each } i = 1, \ldots, N
\end{aligned}
$$

Now a fixed point problem in terms of EV rather than V.

See `L9.py`.

Dynamic programming: Continuous-state example

- Time is discrete, $t = 0, 1, 2, \ldots$
- Discount factor: δ
- Continuous state: $x \in \mathcal{X}$
- Binary choice: $a \in \{0, 1\}$
- Static payoff: $u_a(x) + \epsilon_a$
- Transition process: $g(x_{t+1} \mid x_t = x, a_t = a)$ is a pdf on $\mathcal{X}$

Dynamic programming: Continuous-state example

New challenge

Previously, we had a fixed-point problem in *vector* EV. Now it's a fixed-point problem in *function* EV:

$$\begin{aligned} EV(x) &= \frac{1}{\theta} \log \left[\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right) \right] \\ &= \frac{1}{\theta} \log \left[\sum_{a \in \{0,1\}} \exp\left(\theta \left[u_a(x) + \delta \int EV(x') g(x' \mid x, a) dx' \right] \right) \right] \end{aligned}$$

Challenge: EV is now a function over a continuous domain rather than one over a discrete domain (i.e., no longer a vector).

Dynamic programming: Continuous-state example

New challenge

Previously, we had a fixed-point problem in *vector* EV. Now it's a fixed-point problem in *function* EV:

$$\begin{aligned} EV(x) &= \frac{1}{\theta} \log \left[\exp\left(\theta v_0(x)\right) + \exp\left(\theta v_1(x)\right) \right] \\ &= \frac{1}{\theta} \log \left[\sum_{a \in \{0,1\}} \exp\left(\theta \left[u_a(x) + \delta \int EV(x') g(x' \mid x, a) dx' \right] \right) \right] \end{aligned}$$

Challenge: EV is now a function over a continuous domain rather than one over a discrete domain (i.e., no longer a vector).

Question: How can we solve for EV in this continuous-state case?

Dynamic programming: Continuous-state example

Question: How can we solve for EV in this continuous-state case?

Key ideas:

- A function can be thought of like a vector with infinite length.
- We can't do numerical computations with an infinite-dimensional vector, but what if we solve for $EV(x)$ for a finite set $\{x_1, x_2, \ldots, x_n\} \subset \mathcal{X}$? Is that possible?

Dynamic programming: Continuous-state example

Question: How can we solve for EV in this continuous-state case?

Key ideas:

- A function can be thought of like a vector with infinite length.
- We can't do numerical computations with an infinite-dimensional vector, but what if we solve for $EV(x)$ for a finite set $\{x_1, x_2, \ldots, x_n\} \subset \mathcal{X}$? Is that possible?
  - → But to evaluate the integral, we need to be able to evaluate $EV(x)$ for all x, not just those in our finite set....
  - See `L9.py`

Dynamic programming: Continuous-state example

Optimal stopping problem

- Time is discrete, $t = 0, 1, 2, \ldots$
- Discount factor: δ
- Continuous state: $x \in [0, \infty)$
- Binary choice: $a \in \{0, 1\}$
- Idiosyncratic shocks: $\epsilon_{0t}, \epsilon_{1t} \sim$ iid Extreme Value(θ) with $\theta = 2 \times 10^{-4}$
- Static payoff:

$$u_0(x, \epsilon) = -x + \epsilon_0$$
$$u_1(x, \epsilon) = -10^5 + \epsilon_1$$

- Transition process:
  - $x_{t+1} = x_t + \eta$ where $\eta \sim \chi^2_1$ if $a_t = 0$
  - $x_{t+1} = 0$ if $a_t = 1$

Dynamic programming: Continuous-state example

Optimal stopping problem

- Time is discrete, $t = 0, 1, 2, \ldots$
- Discount factor: δ
- Continuous state: $x \in [0, \infty)$
- Binary choice: $a \in \{0, 1\}$
- Idiosyncratic shocks: $\epsilon_{0t}, \epsilon_{1t} \sim$ iid Extreme Value(θ) with $\theta = 2 \times 10^{-4}$
- Static payoff:

$$u_0(x, \epsilon) = -x + \epsilon_0$$
$$u_1(x, \epsilon) = -10^5 + \epsilon_1$$

- Transition process:
  - $x_{t+1} = x_t + \eta$ where $\eta \sim \chi^2_1$ if $a_t = 0$
  - $x_{t+1} = 0$ if $a_t = 1$

Question: What is an economic interpretation we could give to this problem?

Dynamic programming: Continuous-state example

Optimal stopping problem

- Time is discrete, $t = 0, 1, 2, \ldots$
- Discount factor: δ
- Continuous state: $x \in [0, \infty)$
- Binary choice: $a \in \{0, 1\}$
- Idiosyncratic shocks: $\epsilon_{0t}, \epsilon_{1t} \sim$ iid Extreme Value(θ) with $\theta = 2 \times 10^{-4}$
- Static payoff:

$$u_0(x, \epsilon) = -x + \epsilon_0$$
$$u_1(x, \epsilon) = -10^5 + \epsilon_1$$

- Transition process:
  - $x_{t+1} = x_t + \eta$ where $\eta \sim \chi^2_1$ if $a_t = 0$
  - $x_{t+1} = 0$ if $a_t = 1$

Question: What is an economic interpretation we could give to this problem?

See `L9.py`